

Complete Linux VPS Deployment Guide

Full-Stack Node.js + React + PostgreSQL + Nginx Setup

Last Updated: March 2026
Target Stack: Node.js (Express), React (Vite), PostgreSQL, Nginx, PM2, SSL
Deployment Location: `/var/www/myapp`

Table of Contents

- 1. [Initial VPS Setup](#)
- 2. [Environment & Dependencies](#)
- 3. [PostgreSQL Setup](#)
- 4. [Node.js Backend Deployment](#)
- 5. [React Frontend Deployment](#)
- 6. [Nginx Configuration](#)
- 7. [SSL Certificates \(Let's Encrypt\)](#)
- 8. [PM2 Process Management](#)
- 9. [Monitoring & Logging](#)
- 10. [Troubleshooting](#)
- 11. [Backup & Recovery](#)

1. Initial VPS Setup

SSH Into VPS

```
ssh root@your_vps_ip
```

Update System

```
# Update package lists
apt update

# Upgrade all packages
apt upgrade -y

# Install essential tools
apt install -y curl wget git nano htop net-tools build-essential
```

Create Non-Root User (Recommended)

```
# Create user
adduser deployuser

# Add to sudo group
usermod -aG sudo deployuser

# Switch to new user
su - deployuser
```

Set Up Firewall

```
# Enable UFW firewall
ufw enable

# Allow SSH
ufw allow 22/tcp

# Allow HTTP
ufw allow 80/tcp

# Allow HTTPS
ufw allow 443/tcp

# Check status
ufw status
```

2. Environment & Dependencies

Install Node.js (v20 LTS)

```
# Add NodeSource repository
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

# Install Node.js and npm
apt install -y nodejs

# Verify installation
node --version
npm --version
```

Install Git

```
apt install -y git

# Configure git
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Install PM2 Globally

```
# Install PM2 (process manager)
npm install -g pm2

# Enable startup script
pm2 startup
pm2 save
```

Install Nginx

```
apt install -y nginx

# Start nginx
systemctl start nginx

# Enable on boot
systemctl enable nginx

# Check status
systemctl status nginx
```

3. PostgreSQL Setup

Install PostgreSQL

```
# Add PostgreSQL repository
sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'

wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc | apt-key add -

apt update

# Install PostgreSQL 15 (or latest)
apt install -y postgresql postgresql-contrib

# Start PostgreSQL
systemctl start postgresql

# Enable on boot
systemctl enable postgresql

# Check status
systemctl status postgresql
```

Create Database and User

```
# Switch to postgres user
sudo -u postgres psql

# Create new database
CREATE DATABASE appdb;

# Create new user with password
CREATE USER appuser WITH PASSWORD 'your_secure_password_here';

# Grant privileges
ALTER ROLE appuser SET client_encoding TO 'utf8';
ALTER ROLE appuser SET default_transaction_isolation TO 'read committed';
ALTER ROLE appuser SET default_transaction_deferrable TO on;
ALTER ROLE appuser SET default_transaction_level TO 'committed';
GRANT ALL PRIVILEGES ON DATABASE appdb TO appuser;

# Exit psql
\q
```

Test PostgreSQL Connection

```
# Connect as appuser
psql -U appuser -d appdb -h 127.0.0.1
# Enter password when prompted

# Verify connection
\dt

# Exit
\q
```

Create Application Tables

```
# Connect to database
psql -U appuser -d appdb -h 127.0.0.1

# Create users table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    cartdata TEXT DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

# Create orders table
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    userid INTEGER NOT NULL REFERENCES users(id),
    items TEXT NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
    address TEXT NOT NULL,
    status VARCHAR(50) DEFAULT 'Pending',
    paymentmethod VARCHAR(50),
    payment BOOLEAN DEFAULT FALSE,
    date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    razorpay_order_id VARCHAR(255),
    razorpay_payment_id VARCHAR(255),
    razorpay_signature VARCHAR(255),
    refund_status VARCHAR(50),
    cancel_reason TEXT
);

# Create refund_logs table
CREATE TABLE refund_logs (
    id SERIAL PRIMARY KEY,
    order_id INTEGER NOT NULL REFERENCES orders(id),
    payment_id VARCHAR(255) NOT NULL,
    refund_id VARCHAR(255),
    amount DECIMAL(10, 2),
    status VARCHAR(50),
    type VARCHAR(50),
    reason TEXT,
    date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

# Verify tables created
\dt

# Exit
\q
```

4. Node.js Backend Deployment

Create Application Directory

```
# Create project structure
mkdir -p /var/www/myapp

# Navigate to project directory
cd /var/www/myapp
```

Clone Backend Repository

```
# Clone your backend repo
git clone https://github.com/your-username/your-backend-repo.git

# Navigate to server directory
cd your-backend-repo/server

# Or if your backend is directly in the repo:
cd /var/www/myapp
```

Install Dependencies

```
# Navigate to server directory
cd /var/www/myapp/threads_v1/server

# Install npm packages
npm install
```

Configure Environment Variables

```
# Create .env file
nano .env
```

Sample .env content:

```
# Database
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=appdb
DB_USER=appuser
DB_PASSWORD=your_secure_password_here

# Server
PORT=4000
NODE_ENV=production

# JWT
JWT_SECRET=your_super_secret_jwt_key_change_this

# Email
EMAIL_USER=your_email@gmail.com
EMAIL_PASSWORD=your_app_specific_password
ADMIN_EMAIL=admin@example.com

# Razorpay
RAZORPAY_KEY_ID=rzp_live_xxxxxxxxxxx
RAZORPAY_KEY_SECRET=your_live_secret_key
RAZORPAY_WEBHOOK_SECRET=your_webhook_secret

# Cloudinary
CLOUDINARY_NAME=your_cloudinary_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# Frontend URL
FRONTEND_URL=https://yourdomain.com
```

Save with `Ctrl+X`, then `Y`, then `Enter`.

Verify Backend Starts

```
# Start backend in foreground (test mode)
node server.js

# Should see:
# PostgreSQL Connected
# Server started on PORT: 4000

# Stop with Ctrl+C
```

5. React Frontend Deployment

Clone Frontend Repository

```
# Navigate to myapp directory
cd /var/www/myapp

# Clone frontend repo
git clone https://github.com/your-username/your-frontend-repo.git frontend

# Navigate to frontend
cd frontend
```

Install Dependencies

```
npm install
```

Configure Environment Variables

```
# Create .env file
nano .env
```

Sample `.env` content:

```
VITE_API_URL=https://api.yourdomain.com
VITE_RAZORPAY_KEY_ID=rzp_live_xxxxxxxxxxxx
NODE_ENV=production
```

Create `vercel.json` for SPA Routing + Security Headers

```
nano vercel.json
```

Content:

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ],
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        {
          "key": "Cross-Origin-Opener-Policy",
          "value": "unsafe-none"
        },
        {
          "key": "Cross-Origin-Embedder-Policy",
          "value": "unsafe-none"
        }
      ]
    }
  ]
}
```

Build React Application

```
# Build optimized production bundle
npm run build

# Output will be in dist/ directory
ls dist/
```

Copy Built Files to Nginx

```
# Copy dist to nginx public directory
sudo cp -r dist/* /var/www/yourdomain.com/html/

# Or if serving from custom location:
sudo mkdir -p /var/www/yourdomain.com/html
sudo cp -r dist/* /var/www/yourdomain.com/html/
```

6. Nginx Configuration

Create Nginx Server Block

```
# Create new server block config
sudo nano /etc/nginx/sites-available/yourdomain.com
```

Content (before SSL):


```

# Upstream backend
upstream backend {
    server 127.0.0.1:4000;
}

# HTTP redirect to HTTPS (add after SSL setup)
# server {
#     listen 80;
#     server_name yourdomain.com www.yourdomain.com;
#     return 301 https://$server_name$request_uri;
# }

# Frontend server
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    root /var/www/yourdomain.com/html;
    index index.html;

    # Frontend static files
    location / {
        try_files $uri $uri/ /index.html;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    # Static assets (JS, CSS, images)
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}

# API backend server
server {
    listen 80;
    server_name api.yourdomain.com;

    location / {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;

        # Increase timeouts for webhook processing
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }
}

```

Enable Nginx Site

```
# Enable the site
sudo ln -s /etc/nginx/sites-available/yourdomain.com /etc/nginx/sites-enabled/

# Test nginx config
sudo nginx -t

# Reload nginx
sudo systemctl reload nginx
```

Create Document Root

```
# Create directory for frontend
sudo mkdir -p /var/www/yourdomain.com/html

# Set permissions
sudo chown -R $USER:$USER /var/www/yourdomain.com/html
sudo chmod -R 755 /var/www/yourdomain.com

# Copy frontend dist files
cp -r /var/www/myapp/frontend/dist/* /var/www/yourdomain.com/html/
```

7. SSL Certificates (Let's Encrypt)

Install Certbot

```
apt install -y certbot python3-certbot-nginx
```

Generate SSL Certificate

```
# Generate certificate for both domains
sudo certbot certonly --nginx -d yourdomain.com -d www.yourdomain.com -d api.yourdomain.com

# Follow the prompts and enter your email
```

Update Nginx Configuration with SSL

```
# Edit nginx config
sudo nano /etc/nginx/sites-available/yourdomain.com
```

Updated content with SSL:

```
# Upstream backend
upstream backend {
    server 127.0.0.1:4000;
}

# HTTP to HTTPS redirect
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com api.yourdomain.com;
    return 301 https://$server_name$request_uri;
}

# Frontend HTTPS server
server {
    listen 443 ssl http2;
    server name vourdomain.com www.vourdomain.com;
```

```

# SSL certificates
ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

# SSL security headers
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;
ssl_prefer_server_ciphers on;

# HSTS header
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;

root /var/www/yourdomain.com/html;
index index.html;

# Frontend routing
location / {
    try_files $uri $uri/ /index.html;
    expires 30d;
    add_header Cache-Control "public, immutable";
}

# Static assets
location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
}

# API HTTPS server
server {
    listen 443 ssl http2;
    server_name api.yourdomain.com;

    # SSL certificates
    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # SSL security headers
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    location / {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;

        # Webhook timeout
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }
}

```

Test and Reload Nginx

```
# Test configuration
sudo nginx -t

# Reload nginx
sudo systemctl reload nginx
```

Auto-Renew SSL Certificate

```
# Test renewal
sudo certbot renew --dry-run

# Set up auto-renewal cron job (automatic)
sudo systemctl enable certbot.timer
sudo systemctl start certbot.timer

# Verify
sudo systemctl status certbot.timer
```

8. PM2 Process Management

Start Backend with PM2

```
# Navigate to backend
cd /var/www/myapp/threads_v1/server

# Start with PM2
pm2 start server.js --name "myapp"

# Save PM2 configuration
pm2 save

# Enable startup script
pm2 startup
pm2 save
```

Create PM2 Ecosystem File (Optional but Recommended)

```
# Create ecosystem.config.js in server directory
cd /var/www/myapp/threads_v1/server
nano ecosystem.config.js
```

Content:

```
module.exports = {
  apps: [{
    name: 'myapp',
    script: './server.js',
    instances: 'max',
    exec_mode: 'cluster',
    env: {
      NODE_ENV: 'production',
      PORT: 4000
    },
    error_file: './logs/error.log',
    out_file: './logs/out.log',
    log_date_format: 'YYYY-MM-DD HH:mm:ss Z',
    merge_logs: true,
    autorestart: true,
    max_memory_restart: '1G',
    watch: false,
    ignore_watch: ['node_modules', 'logs', '.git']
  }]
};
```

Start with Ecosystem File

```
# Start using ecosystem file
pm2 start ecosystem.config.js

# Save configuration
pm2 save
```

PM2 Commands Reference

```
# List all processes
pm2 list

# View logs
pm2 logs myapp

# Watch logs in real-time
pm2 logs myapp --lines 100 --follow

# Restart process
pm2 restart myapp

# Stop process
pm2 stop myapp

# Delete process
pm2 delete myapp

# Restart all processes
pm2 restart all

# View process details
pm2 info myapp

# Monitor CPU and memory
pm2 monit
```

9. Monitoring & Logging

Check Application Health

```
# Test frontend
curl -I https://yourdomain.com

# Test API
curl https://api.yourdomain.com/health

# Check backend logs
pm2 logs myapp

# Check nginx logs
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log

# Check PostgreSQL logs
sudo tail -f /var/log/postgresql/postgresql.log
```

Create Log Directories for PM2

```
# Create logs directory
mkdir -p /var/www/myapp/threads_v1/server/logs

# Set permissions
chmod 755 /var/www/myapp/threads_v1/server/logs
```

Monitor Disk Space

```
# Check disk usage
df -h

# Check inodes
df -i

# Check specific directory size
du -sh /var/www/myapp

# Find large files
find /var/www/myapp -type f -size +100M
```

Monitor Memory and CPU

```
# Interactive monitoring
htop

# Quick check
free -h
ps aux | grep node
```

10. Troubleshooting

Backend Not Starting

```
# Check if port 4000 is already in use
ss -tlnp | grep 4000

# Kill process on port 4000
sudo lsof -ti:4000 | xargs kill -9

# Kill all node processes
pkill -f node

# Check error logs
pm2 logs myapp

# Test database connection
psql -U appuser -d appdb -h 127.0.0.1
```

Nginx Returns 502 Bad Gateway

```
# Verify backend is running
pm2 list

# Check if backend listening on correct port
netstat -tlnp | grep 4000

# Restart backend
pm2 restart myapp

# Check nginx config
sudo nginx -t

# Reload nginx
sudo systemctl reload nginx
```

PostgreSQL Connection Issues

```
# Check if PostgreSQL is running
systemctl status postgresql

# Verify connection with correct credentials
psql -U appuser -d appdb -h 127.0.0.1 -W

# Check PostgreSQL logs
sudo tail -f /var/log/postgresql/postgresql.log

# Verify database exists
sudo -u postgres psql -l

# Recreate user if needed
sudo -u postgres psql
DROP USER IF EXISTS appuser;
CREATE USER appuser WITH PASSWORD 'new_password';
GRANT ALL PRIVILEGES ON DATABASE appdb TO appuser;
\q
```

SSL Certificate Issues

```
# Check certificate validity
sudo certbot certificates

# Renew certificate manually
sudo certbot renew

# Test renewal
sudo certbot renew --dry-run

# Check Nginx SSL
sudo nginx -T | grep -i ssl
```

Disk Space Issues

```
# Clean npm cache
npm cache clean --force

# Remove old logs
find /var/www -name "*.log" -type f -mtime +30 -delete

# Clean PM2 logs
pm2 flush

# Remove node_modules and reinstall (if needed)
cd /var/www/myapp/threads_v1/server
rm -rf node_modules
npm install --production
```

11. Backup & Recovery

Backup Database

```
# Full database backup
sudo -u postgres pg_dump -U appuser appdb > ~/appdb_backup_$(date +%Y%m%d_%H%M%S).sql

# Compressed backup
sudo -u postgres pg_dump -U appuser appdb | gzip > ~/appdb_backup_$(date +%Y%m%d_%H%M%S).sql.gz

# List backups
ls -lh ~/.sql.gz
```

Restore Database

```
# From SQL file
psql -U appuser -d appdb -h 127.0.0.1 < appdb_backup_20260324_143022.sql

# From compressed backup
gunzip < appdb_backup_20260324_143022.sql.gz | psql -U appuser -d appdb -h 127.0.0.1
```

Backup Application Code

```
# Backup entire application
tar -czf ~/myapp_backup_$(date +%Y%m%d_%H%M%S).tar.gz /var/www/myapp/

# List backups
ls -lh ~/*.tar.gz

# Restore from backup
tar -xzf myapp_backup_20260324_143022.tar.gz -C /
```

Automated Daily Backup

```
# Create backup script
nano ~/backup.sh
```

Content:

```
#!/bin/bash

BACKUP_DIR="$HOME/backups"
BACKUP_DATE=$(date +%Y%m%d_%H%M%S)
DB_NAME="appdb"
DB_USER="appuser"

# Create backup directory
mkdir -p "$BACKUP_DIR"

# Backup database
sudo -u postgres pg_dump -U "$DB_USER" "$DB_NAME" | gzip > "$BACKUP_DIR/db_backup_$BACKUP_DATE.sql.gz"

# Backup application code
tar -czf "$BACKUP_DIR/app_backup_$BACKUP_DATE.tar.gz" /var/www/myapp/

# Keep only last 7 days of backups
find "$BACKUP_DIR" -name "*.gz" -mtime +7 -delete

echo "Backup completed at $BACKUP_DATE"
```

Schedule Automated Backups

```
# Make script executable
chmod +x ~/backup.sh

# Add to crontab (runs daily at 2 AM)
crontab -e

# Add this line:
0 2 * * * /root/backup.sh >> /var/log/backup.log 2>&1

# Verify cron job
crontab -l
```

Deployment Checklist

Pre-Deployment

- ☐ VPS created and accessible
- ☐ SSH keys configured
- ☐ Firewall rules set up

- ☐ Domain DNS pointed to VPS IP
- ☐ SSL certificate generated

Backend Setup

- ☐ Node.js v20 installed
- ☐ Git repository cloned
- ☐ Dependencies installed (`npm install`)
- ☐ `.env` file created with all variables
- ☐ Database created and tables initialized
- ☐ Backend tested locally (`node server.js`)
- ☐ PM2 configured and started
- ☐ Backend accessible on port 4000

Frontend Setup

- ☐ React dependencies installed (`npm install`)
- ☐ `.env` file configured with API URL
- ☐ Build created (`npm run build`)
- ☐ `vercel.json` configured with headers and routing
- ☐ Built files copied to Nginx directory
- ☐ Frontend accessible on HTTPS

Nginx Setup

- ☐ Server blocks created for both domains
- ☐ SSL certificates configured
- ☐ Nginx config tested and reloaded
- ☐ Both frontend and API responding on HTTPS
- ☐ HSTS headers in place

Monitoring

- ☐ PM2 logs checked for errors
- ☐ Nginx access/error logs monitored
- ☐ Database connection verified
- ☐ Disk space monitored
- ☐ Backup strategy implemented
- ☐ SSL certificate renewal automated

Quick Reference Commands

```
# Backend management
pm2 start myapp          # Start
pm2 restart myapp        # Restart
pm2 stop myapp           # Stop
pm2 logs myapp           # View logs
pm2 list                 # List all processes

# Nginx management
sudo systemctl start nginx # Start
sudo systemctl restart nginx # Restart
sudo systemctl reload nginx # Reload (no downtime)
sudo systemctl stop nginx  # Stop
sudo nginx -t              # Test config

# Database
psql -U appuser -d appdb -h 127.0.0.1 # Connect
\dt                                   # List tables
\q                                   # Exit

# SSL
sudo certbot renew          # Renew certificates
sudo certbot certificates  # List certificates

# System
df -h                        # Disk usage
free -h                    # Memory usage
top                          # Process monitor
htop                       # Interactive process monitor

# Logs
tail -f /var/log/nginx/access.log    # Nginx access
tail -f /var/log/nginx/error.log     # Nginx errors
sudo tail -f /var/log/postgresql/*   # PostgreSQL logs
pm2 logs myapp                      # Application logs
```

Security Best Practices

1. Use Strong Passwords

- PostgreSQL: Complex mix of uppercase, lowercase, numbers, symbols
- JWT_SECRET: At least 32 random characters

2. Environment Variables

- Never commit `.env` to git
- Use `.gitignore` to exclude sensitive files
- Rotate secrets periodically

3. Firewall

- Only open necessary ports (22, 80, 443)
- Use UFW rules to restrict access
- Consider IP whitelisting for admin access

4. SSL/TLS

- Always use HTTPS in production
- Enable HSTS headers
- Keep certificates updated

5. Database

- Use non-root user for application
- Regular backups
- Monitor for unusual activity

6. Application

- Input validation
- SQL injection prevention (use parameterized queries)
- Rate limiting
- CORS properly configured

- Helmet.js for security headers

7. Updates

- Keep OS packages updated
- Update Node.js and npm regularly
- Update package dependencies
- Monitor security advisories

Additional Resources

- [Node.js Documentation](#)
 - [Express.js Guide](#)
 - [PostgreSQL Documentation](#)
 - [Nginx Documentation](#)
 - [PM2 Documentation](#)
 - [Let's Encrypt](#)
 - [Linux Academy](#)
-

Last Updated: March 17, 2026

Author: SAI SWAROOP PENTAKOTA

Version: 1.0